Sistema de Control de Inventario

Informe de diseño y fundamentación

Trabajo Práctico Integrador - Programación 1

Tecnicatura Universitaria en Programación a Distancia

Universidad Tecnológica Nacional (UTN)

Alumna: Ludmila García

Archivo que se ejecuta: control_inventario.py

Video Demostrativo: https://youtu.be/N2488aV8hZs

1. De qué trata el trabajo

Una ferretería necesita dejar de llevar su stock "a mano" para no perder control de las herramientas que tiene. Para eso se hizo un programa en Python que funciona con un menú de siete opciones: cargar herramientas, ver el inventario, consultar el stock de una herramienta, listar las agotadas, dar de alta una herramienta nueva, registrar ventas o ingresos de mercadería y salir.

Los datos se guardan en una sola estructura: una **lista de diccionarios** llamada `inventario`. Se actualizan en el momento, mientras el programa está abierto. Cuando se cierra el programa, los datos se pierden: el trabajo no pide guardarlos en un archivo.

Además del código, este informe explica **cómo se pensó** la solución y **por qué** se tomaron las decisiones, con un diagrama del flujo de las operaciones principales.

2. Lenguaje, entorno y recursos de desarrollo

Python (lenguaje)

Python es un lenguaje de programación de alto nivel, muy usado para aprender porque su sintaxis es clara y se parece bastante a leer en inglés. Se usó la **versión 3.14.6**, en la que se probó todo el programa. Se eligió porque es el lenguaje que se enseña en la materia, porque trae incorporados lo que hace falta para este trabajo (listas, diccionarios, funciones, `try/except` y `raise`) y porque no requiere instalar librerías adicionales: el programa usa solamente lo que viene con Python.

Visual Studio Code (entorno)

Visual Studio Code es un editor de código gratuito de Microsoft. Se usó la **versión 1.145.0**. Se eligió porque resalta con colores la sintaxis de Python, marca errores de escritura, permite ejecutar el programa desde la terminal integrada y es el entorno que pide el enunciado para grabar el video.

Apuntes y resúmenes de la materia

Como material de consulta se usaron los apuntes y resúmenes propios de las 10 unidades de la materia. De ahí se tomaron los temas que se aplican en el programa: listas, diccionarios, funciones, condicionales, bucles y manejo de errores con `try/except` y `raise`. Se eligieron porque son el material que se estudió en la cursada y están alineados con lo que pide el trabajo.

Cómo se ejecuta

El programa está en **un solo archivo**: `control_inventario.py`. Para iniciarlo se ejecuta ese archivo (por ejemplo, con `python control_inventario.py` en la terminal).

Draw.io (diagramas de flujo)

Draw.io, también llamado diagrams.net, es una herramienta gratuita para crear diagramas desde el navegador o desde su versión de escritorio. Se usó la versión 32.6.10 para armar los diagramas de flujo de la sección 5 (menú principal, carga de herramientas, alta de producto y actualización de stock). Se eligió porque es gratuita, no requiere instalar nada, trae las figuras estándar de los diagramas de flujo (inicio y fin, procesos y decisiones) y permite usar colores para distinguir los errores de los cambios efectivos de datos.

3. Diseño de la solución

Antes de escribir código se decidió dividir el programa en funciones chicas, cada una con una sola tarea. Así, si algo falla en la consulta de stock, se sabe en qué función buscar. Hay una función por cada opción del menú (de la 1 a la 6) y varias funciones auxiliares que se encargan de validar datos y buscar herramientas. La opción 7 se resuelve en `main()`, cuando termina el bucle del menú.

Función	Qué hace
main()	Crea `inventario = []`, repite el menú con un `while` y atrapa los errores con `try/except`. Termina con la opción 7.
mostrar_menu()	Imprime las siete opciones.
cargar_herramientas(inventario)	Opción 1. Carga varias herramientas. Si un dato es inválido, lo captura y vuelve a pedir ese dato.
mostrar_inventario(inventario)	Opción 2. Recorre la lista y muestra cada herramienta con su stock.
consultar_stock(inventario)	Opción 3. Busca una herramienta por nombre y muestra su stock.
reporte_agotados(inventario)	Opción 4. Muestra solo las herramientas con stock igual a 0.
alta_producto(inventario)	Opción 5. Agrega una herramienta nueva al final de la lista.
actualizar_stock(inventario)	Opción 6. Registra una venta (resta) o un ingreso (suma).
convertir_entero(texto, dato)	Pasa un texto a número entero; si no se puede, lanza un error con un mensaje claro.
validar_mayor_que_cero(texto, dato)	Pide que el entero sea mayor que 0 (cantidad a cargar, unidades).
validar_stock(texto)	Pide que el stock sea un entero mayor o igual a 0.
limpiar_nombre(texto)	Saca los espacios de los extremos; si queda vacío, lanza un error.
buscar_herramienta(inventario, nombre)	Busca sin importar mayúsculas ni espacios; devuelve la herramienta o None.
validar_nombre_nuevo(inventario, texto)	Valida que el nombre no esté vacío ni repetido.

Decisiones importantes:

1) **Una sola estructura de datos.** Todo el inventario vive en la lista de diccionarios. Los resultados (como el reporte de agotados) se obtienen recorriendo esa lista, sin crear listas ni diccionarios auxiliares.

2) **Sin variables globales.** `inventario` se crea dentro de `main()` y se pasa como parámetro a cada función. Como las listas son mutables, la función recibe la misma lista (no una copia): un `append` o un cambio de stock hecho dentro de la función se ve después en `main()`, y por eso no hace falta devolver la lista con `return`.

3) **Dos formas de manejar los errores**, según la opción. En la opción 1 el error se captura dentro de la misma función para volver a pedir el dato sin perder lo ya cargado. En las opciones 3, 5 y 6 el error sale de la función y lo atrapa el `except` de `main()`, que muestra el motivo y vuelve al menú.

4) **Un dato inválido nunca deja datos a medias.** Una herramienta se agrega a la lista recién cuando su nombre y su stock ya fueron validados, y una operación que falla no modifica ningún stock.

4. Conceptos aplicados y dónde aparecen en el programa

4.1 Listas

Una lista es una colección ordenada de elementos que puede crecer. En el programa, `inventario` es una lista que arranca vacía en `main()`. Cada herramienta nueva se agrega al final con `append`, en `cargar_herramientas()` (opción 1) y en `alta_producto()` (opción 5). Para mostrar, buscar o contar herramientas se la recorre con `for producto in inventario`, y se usa `len(inventario)` para saber si todavía está vacía.

```
def main():
    inventario = []   # la lista empieza vacía
    ...
```

4.2 Diccionarios

Un diccionario guarda datos como pares clave-valor. Cada herramienta es un diccionario con **exactamente dos claves**: `'herramienta'` (el nombre) y `'cantidad'` (el stock). Como el enunciado lo exige, el diccionario no se crea suelto: se arma directamente dentro del `append`. En la venta o el ingreso se modifica el valor de la clave `'cantidad'` del diccionario que ya está en la lista.

```
inventario.append({'herramienta': nombre, 'cantidad': stock})
producto['cantidad'] = producto['cantidad'] - unidades
```

4.3 Funciones

Una función es un bloque de código con nombre que hace una tarea y se puede reutilizar. Se usaron para separar responsabilidades: una función por opción del menú y funciones auxiliares para validar. Por ejemplo, `validar_nombre_nuevo()` se reutiliza en las opciones 1 y 5, y `buscar_herramienta()` en las opciones 3 y 6 y en el control de duplicados. Los datos compartidos (`inventario`) entran por parámetro, y las funciones auxiliares que devuelven un resultado usan `return`.

Un ejemplo es `convertir_entero(texto, dato)`, que tiene **dos parámetros** porque cada uno cumple un trabajo distinto. `texto` es lo que escribió el usuario y es el valor que se convierte con `int()`. `dato` es el nombre de lo que se está pidiendo (por ejemplo "Stock" o "Unidades"); no interviene en la conversión, sino que se usa solo para armar el mensaje si el valor no es un número. Gracias a eso, una única función sirve para todos los datos del programa y cada error indica cuál fue el dato mal ingresado, sin repetir código en una función por cada caso.

```
convertir_entero('abc', 'Stock')
   -> error: Stock: se esperaba un número entero y se ingresó abc
convertir_entero('abc', 'Unidades')
   -> error: Unidades: se esperaba un número entero y se ingresó abc
```

4.4 Condicionales

Un condicional (`if / elif / else`) hace que el programa tome un camino u otro según una situación. Se usan para elegir la opción del menú en `main()`, y para los casos informativos que no necesitan una excepción: inventario vacío, herramienta que no existe o tipo de movimiento que no es venta ni ingreso. En la opción 6 el tipo de movimiento se compara como texto, sin convertirlo con `int()`.

```
tipo = input('Tipo de movimiento (1 o venta / 2 o ingreso): ').strip().lower()
if tipo == '1' or tipo == 'venta':
    es_venta = True
elif tipo == '2' or tipo == 'ingreso':
    es_venta = False
else:
    print('Tipo de movimiento inválido: solo se acepta venta o ingreso.')
    return
```

4.5 Bucles

Un bucle repite instrucciones. El `while opcion != 7` de `main()` hace que el menú sea persistente: vuelve a aparecer después de cada operación hasta que se elige salir. En la opción 1, un `for` con `range()` repite la carga tantas veces como herramientas se indicaron, y dentro hay tres `while True` con `break` que vuelven a pedir un dato mientras no sea válido. Para listar o buscar, el `for producto in inventario` recorre la lista; si encuentra la herramienta buscada, la función devuelve ese diccionario y no sigue recorriendo.

4.6 Manejo de errores: try/except y raise

Son dos situaciones distintas y se resuelven con dos herramientas distintas:

try/except sirve para *capturar* un error. Si el usuario escribe "abc" donde va un número, `int()` falla con un `ValueError`. En `convertir_entero()` se captura ese fallo y se lo lanza de nuevo con un mensaje claro. `except` solo atrapa el error: el reintento lo organiza un bucle que vuelve a pedir el dato.

raise sirve para *lanzar* un error cuando el dato es un texto válido pero rompe una regla del negocio: nombre vacío o repetido, stock negativo, cantidades menores o iguales a 0, venta que supera el stock u opción del menú fuera de rango. Se usa `raise ValueError(...)` con un mensaje descriptivo; no hizo falta crear clases propias porque `ValueError` ya viene en Python.

```
def convertir_entero(texto, dato):
    try:
        return int(texto)
    except ValueError:
        raise ValueError(f'{dato}: se esperaba un número entero ...')

def validar_stock(texto):
    numero = convertir_entero(texto, 'Stock')
    if numero < 0:
        raise ValueError('Stock: no puede ser negativo (el mínimo es 0).')
    return numero
```

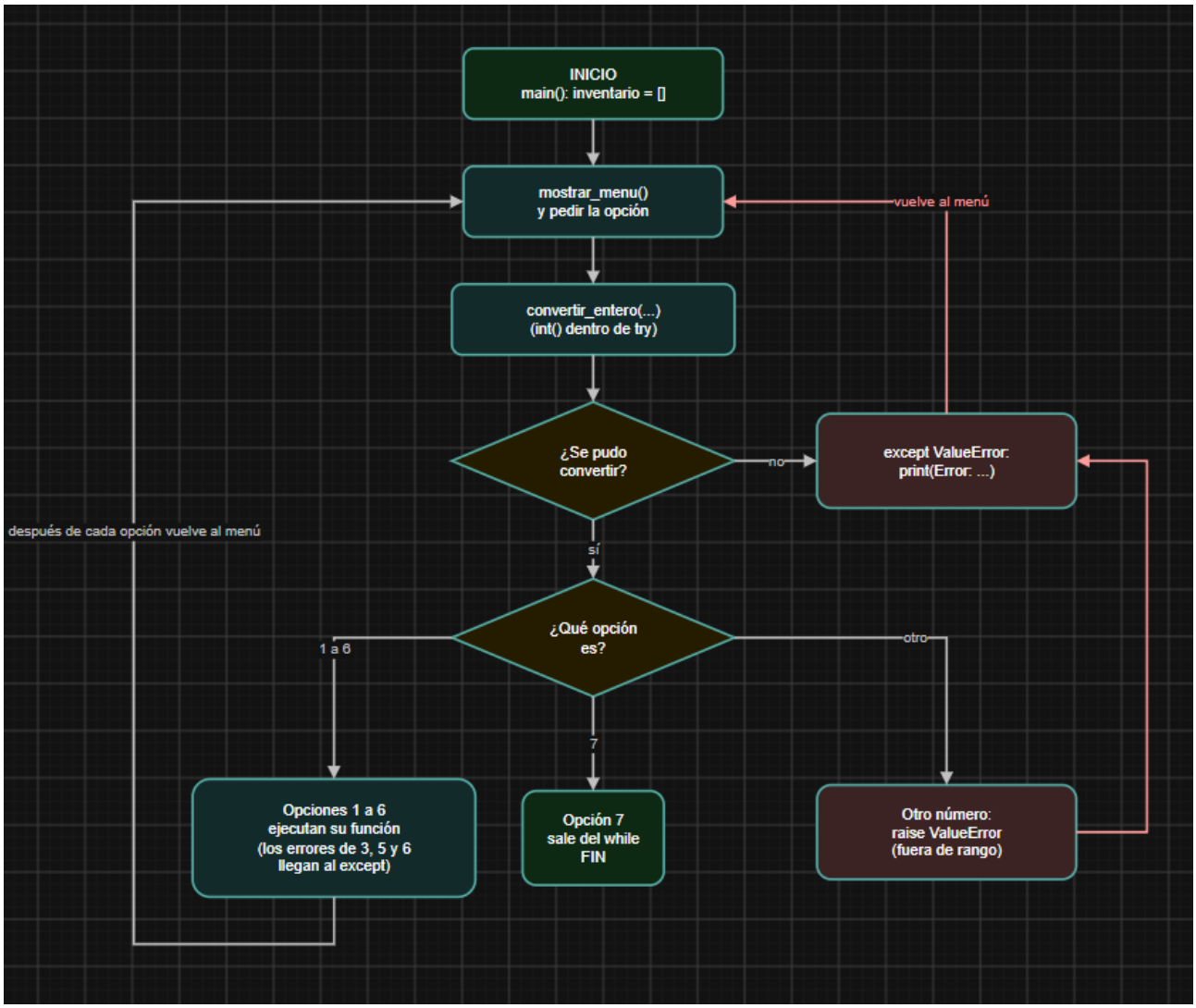
Dónde se captura cada error:

Opción	Dato inválido	Qué pasa
Menú	Texto que no es número, o número fuera de 1 a 7	Lo captura `main()`, muestra el motivo y vuelve a pedir la opción.
1	Cantidad a cargar, nombre o stock inválidos	Se captura dentro de `cargar_herramientas()` y se vuelve a pedir solo ese dato (el nombre válido se conserva).
3	Nombre vacío	Sale de la función; lo atrapa `main()` y vuelve al menú.
5	Cualquier dato inválido	Sale de la función; lo atrapa `main()`; no se agrega nada.
6	Nombre vacío, unidades inválidas o venta mayor al stock	Sale de la función; lo atrapa `main()`; el stock no se modifica.

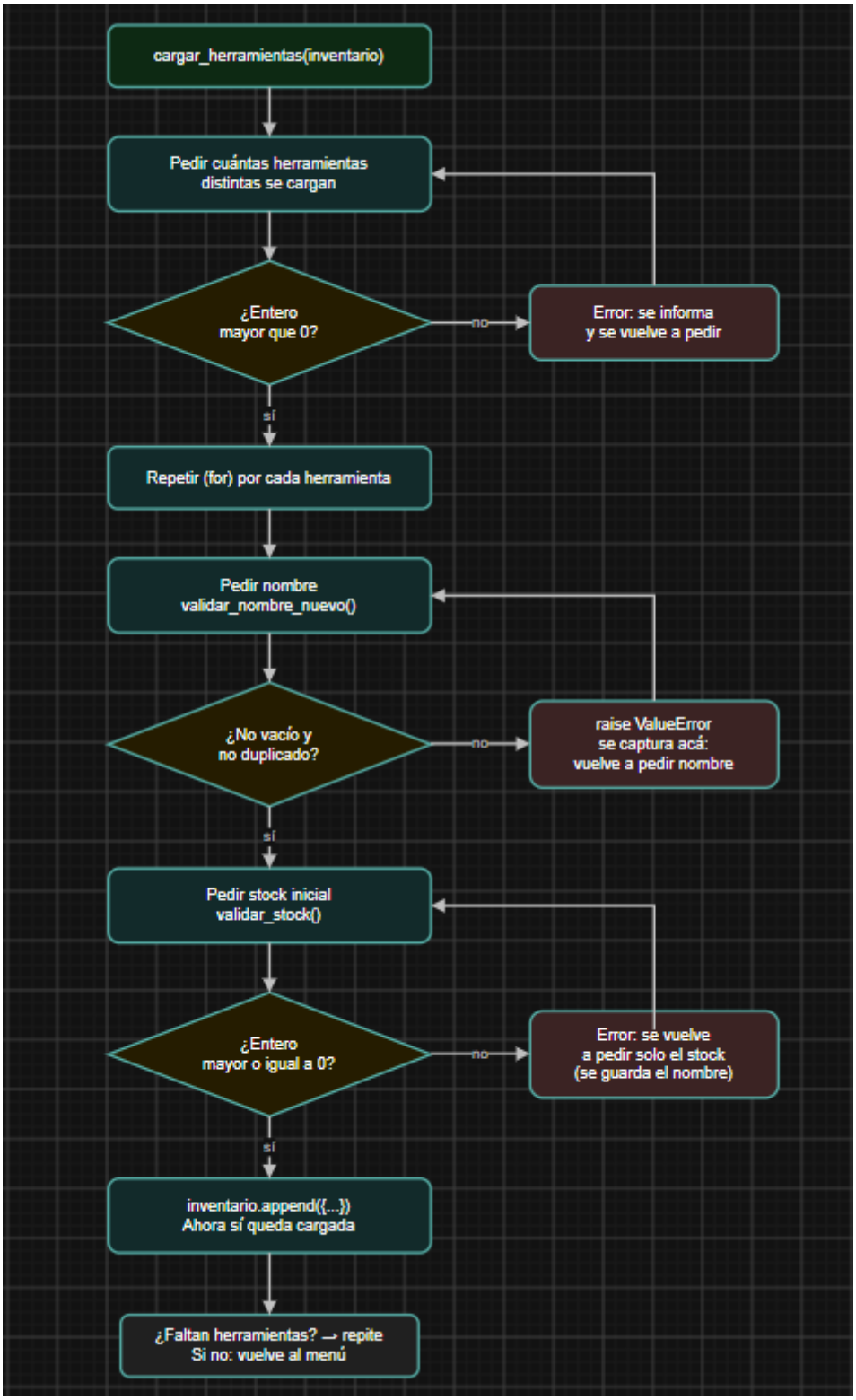
5. Diagramas de flujo

Los siguientes esquemas muestran el recorrido de las operaciones principales: el menú, la carga de herramientas (con su validación), el alta de un producto y la actualización de stock. Los cuadros rojos indican errores y los verdes el inicio, el fin o un cambio efectivo de datos.

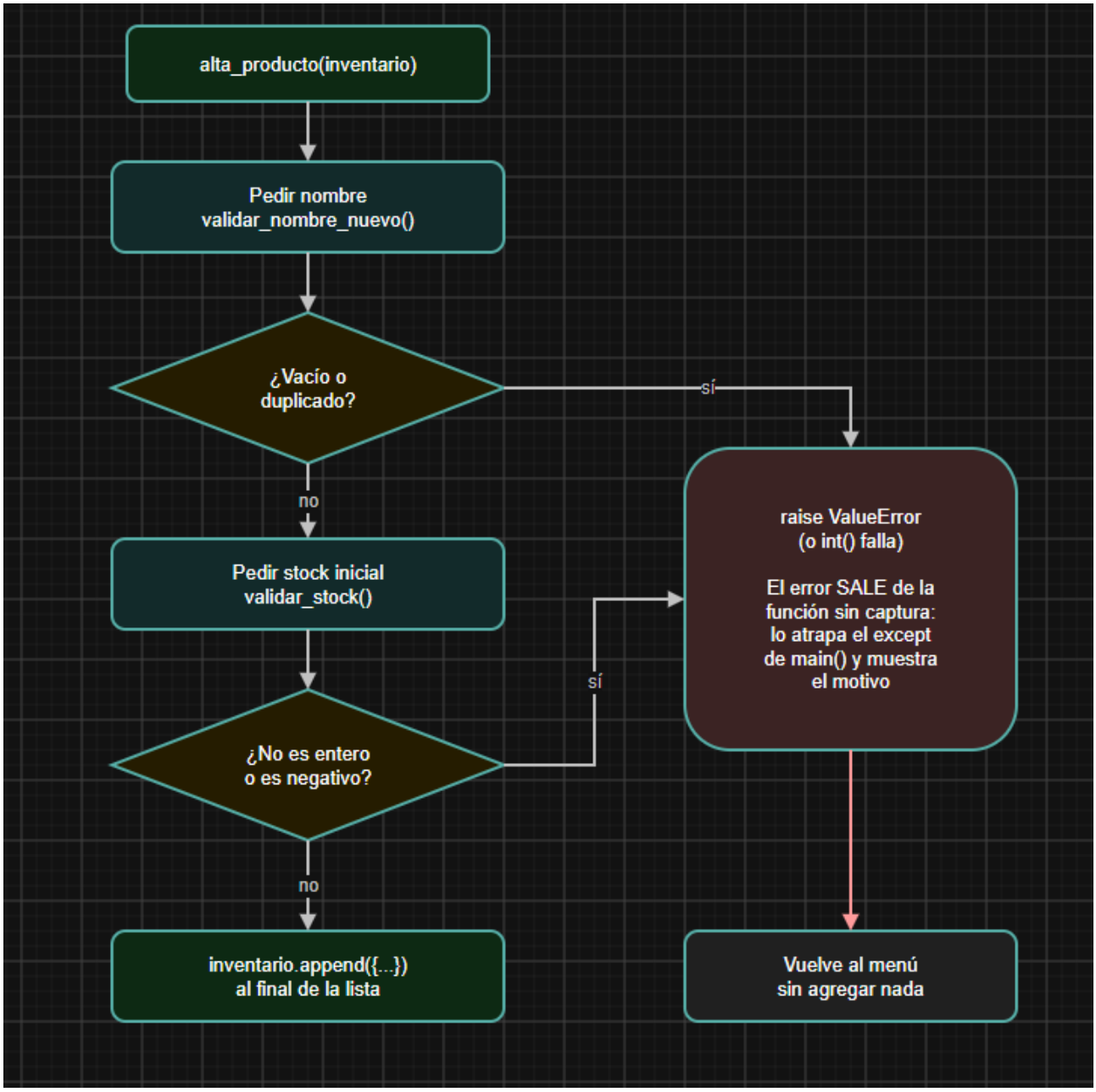
5.1 Menú principal



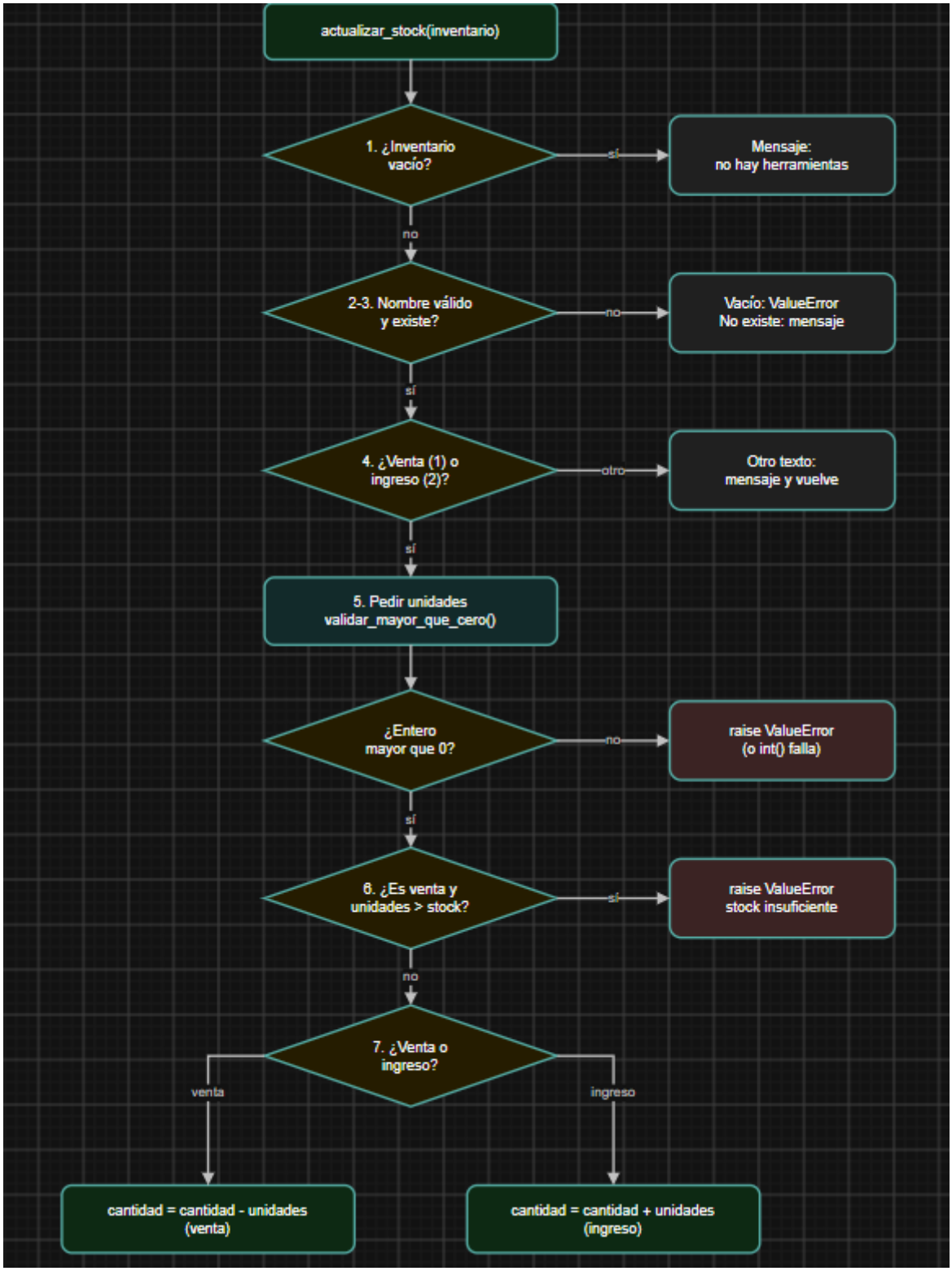
5.2 Carga y validación de datos (opción 1)



5.3 Alta de nuevo producto (opción 5)



5.4 Actualización de stock (opción 6)



En los cuadros rojos y grises el stock no se modifica

6. Pruebas realizadas

Se probó el programa con los casos de la tabla de pruebas manuales del enunciado, partiendo en cada caso del estado inicial indicado. Resumen de los resultados:

Caso probado	Resultado obtenido
Menú: "abc" o número fuera de rango (9, 0, -3)	Muestra el error y vuelve a pedir la opción.
Opción 1: cantidad 0, -1, "abc" o 3.5	Informa el error y vuelve a pedir ese dato.
Opción 1: nombre vacío, de solo espacios o repetido ("martillo", " Martillo ")	Se rechaza y se vuelve a pedir el nombre.
Opción 1: stock -2, "abc" o 3.5	Se vuelve a pedir solo el stock, conservando el nombre; no se agrega nada hasta que sea válido.
Opción 1: stock 0, y segunda ejecución de la opción	El 0 se acepta; la segunda carga conserva lo anterior y agrega las nuevas sin duplicados.
Opción 1: 3 herramientas, con "abc" como stock de la 2.ª	Se pide de nuevo solo ese stock y al final se agregan exactamente 3.
Opciones 2, 3, 4, 5 y 6 con el inventario vacío	Opciones 2, 3, 4 y 6: informan que no hay herramientas (3 y 6 sin pedir datos). La 5 permite dar de alta.
Opción 3: "MARTILLO", " martillo ", nombre vacío, herramienta inexistente	Encuentra la herramienta ignorando mayúsculas y espacios; el vacío muestra el motivo; la inexistente se informa.
Opción 4: con agotados, sin agotados	Muestra solo las de stock 0; si no hay, informa que ninguna tiene stock cero.
Opción 5: nombre vacío, duplicado, stock -2, 3.5, "abc"	Muestra el motivo, vuelve al menú y no agrega nada. Con stock 3 o 0 agrega al final.
Opción 6: herramienta inexistente, nombre vacío, tipo inválido	Informa y vuelve al menú sin modificar el inventario.
Opción 6: unidades 0, -1, "abc" o 3.5 (venta e ingreso)	Rechaza el movimiento y el stock no cambia.
Opción 6: vender 2 de 10, vender 3 de 3, vender 4 de 3, ingresar 2 con stock 0	Queda en 8; queda en 0; error y sigue en 3; queda en 2 y sale del reporte de agotados.

7. Cumplimiento de las reglas técnicas

Regla	Cómo se cumple
2.1 Estructuras permitidas	Solo se usan secuencias, if/elif/else, while, for (con break), def y return, además de try/except y raise. No se usa nada fuera de esa lista.
2.2 Estructura de datos	Una lista de diccionarios llamada inventario, con exactamente las claves 'herramienta' y 'cantidad'.
2.3 Modularización	Las opciones 1 a 6 tienen su propia función; la opción 7 se resuelve en main().
2.4 Manejo de errores	try/except para las conversiones y raise ValueError para las reglas del negocio.
2.5 Prohibiciones	Sin clases, sin diccionarios sueltos (se arman dentro de append) y sin otras colecciones fuera de inventario.
2.6 Variables globales	No hay ninguna: inventario se crea en main() y viaja por parámetro.